

Unit-II Sub queries, Joins, Sequence, Index, Pseudo columns, Views

Subquery: (Inner query or Nested query)

- A query embedded inside another query is called 'Subquery'
- **OR** A query within another query is called as '**subquery**' (or **nested query**).
- In some situation, we have to pass result (i.e. output) of one query as input to main query, in this case 'Sub query' is used.
- Here, main query is also called 'outer query' to which result of sub query is passed as input.
- Note that:
 - The **subquery** executes **first** and returns a result.
 - The **main (outer) query** uses that result to complete execution.
- In Oracle, subqueries are commonly used inside:
 - SELECT clause for returning computed i.e. calculated values
 - WHERE clause for filtering data
 - HAVING clause for filtering groups
 - FROM clause for treating subquery results as a temporary table

❖ Purpose of a Subquery in Oracle:

The main **purpose** of a subquery is to:

1. **Break down complex queries** into smaller, manageable parts.
2. **Provide intermediate results** to be used by the main (outer) query.
3. **Perform comparisons** against aggregated or calculated values.
4. **Filter data dynamically** instead of hardcoding values.
5. **Enhance flexibility** such as subqueries almost used anywhere in WHERE, FROM, HAVING, SELECT clause

❖ Usage of Subqueries in Oracle:

Following are the several usage of subquery:

1. Subquery is used for Filtering rows based on calculations
2. Subquery is used to work with multiple values.
3. Subquery used for filtering groups that uses aggregated data.
4. Subquery used to create virtual tables (Inline Views)
5. Subquery used for Row-by-row comparison (record comparison) (Correlated Subquery)

General Syntax for subquery-

SQL>SELECT col1,col2,... FROM tablename WHERE **column** operator (**SELECT column FROM tablename WHERE condition**);

Here,

Col1,col2,... → columns to display in the main query

Column is to which result of sub query is passed as input.

operator → comparison operators (=, >, <, IN, ANY, ALL, EXISTS)

subquery → placed inside parentheses ()

Example-

SQL>SELECT * FROM employee WHERE salary=(SELECT MIN(salary) FROM employee);

Here,

➤ **Inner Query (Subquery) is:**

SQL>SELECT MIN(salary) FROM employee;

- The database first executes this subquery.
- It scans the entire 'employee' table and finds the **minimum salary**.

➤ **Outer Query (Main Query) is:**

SQL>SELECT * FROM employee WHERE salary = <result of subquery>;

Here, Oracle passes the result of the sub query to main query and retrieves all employees whose salary is equal to the minimum salary.

Types of Subqueries in Oracle:

We can classify subqueries based on **result**, **placement**, and **correlation** which are as follow-

A) Based on Result:

“Based on result” means **how many rows/columns the subquery returns** to the outer query.

- A.1) Single-row subquery
- A.2) Multi-row subquery
- A.3) Multi-column subquery
- A.4) Scaler subquery

B) Based on Placement:

“Based on Placement” **subquery** means **where the subquery is written inside the SQL statement**

- B.1) Subquery in WHERE clause
- B.2) Subquery in FROM clause
- B.3) Subquery in SELECT clause

C) Based on Correlation:

“Based on Correlation” **subquery** means whether the **subquery depends on the outer query** or not.

- C.1) Correlated subquery
- C.2) Non-correlated subquery

Let's see all these types of subqueries in details-

A) Based on Result:

A.1) Single-row subquery:

- A **subquery that returns exactly one row** (but it **may** contain **one or multiple** columns) is called ‘Single-row subquery’
- This subquery used when you need to compare a column with a **single result** from another query.
- If the subquery returns **more than one row**, Oracle will throw an error:
- ORA-01427: single-row subquery returns more than one row

◆ Syntax:

SQL>SELECT column1, column2, ... **FROM** table_name **WHERE** column operator
(**SELECT** single_column **FROM** table_name **WHERE** condition);

Here,

- **operator** → Single-row comparison operators only such as = , > , < , >= , <= , <>

◆ Examples: Consider we have ‘employee’ table as follow-

SQL> SELECT * FROM EMPLOYEE;			
EMP_ID	EMP_NAME	DOJ	SALARY
1	VIJAY	04-AUG-02	10000
2	KIRAN	05-SEP-99	15000
3	RAJESH	25-JUL-95	25000
4	SHANTA	12-MAY-05	18000

1. Compare with Minimum Salary:

Find employees who earn the **minimum salary** in the company.

```
SQL> SELECT * from employee WHERE salary=(SELECT MIN(salary) FROM employee);
```

EMP_ID	EMP_NAME	DOJ	SALARY
1	VIJAY	04-AUG-02	10000

— This Subquery returns **one value** (minimum salary).

2. Compare with Maximum Salary

Find employees who earn **less than the maximum salary**.

```
SQL> SELECT * from employee WHERE salary < (SELECT MAX(salary) FROM employee);
```

EMP_ID	EMP_NAME	DOJ	SALARY
1	VIJAY	04-AUG-02	10000
2	KIRAN	05-SEP-99	15000
4	SHANTA	12-MAY-05	18000

3. Compare with Average Salary

Find employees earning **above the company's average salary**.

```
SQL> SELECT * from employee WHERE salary > (SELECT AVG(salary) FROM employee);
```

EMP_ID	EMP_NAME	DOJ	SALARY
3	RAJESH	25-JUL-95	25000
4	SHANTA	12-MAY-05	18000

4. Using in SELECT Clause (Scalar Single-row Subquery)

Show each employee along with the **highest salary in the company**.

```
SQL> SELECT emp_id,emp_name, salary,(SELECT MAX(salary) FROM employee) AS highest_salary FROM employee;
```

EMP_ID	EMP_NAME	SALARY	HIGHEST_SALARY
1	VIJAY	10000	25000
2	KIRAN	15000	25000
3	RAJESH	25000	25000
4	SHANTA	18000	25000

◆ Key Points of Single-row subquery:

1. It returns only **one row, one column**.
2. It must be used with **single-row operators**.
3. If multiple rows are returned then it gives **error**.
4. It can be used in WHERE, HAVING, and SELECT clauses.

A.2) Multi-row subquery:

- A subquery that returns more than one row but one column is called 'Multi-row subquery'
- This subquery returns multiple values so we **cannot use single-row operators** (=, <, > etc).
- Instead of using single-row operators, we use **multi-row operators** IN, ANY (or SOME), ALL

Syntax:

```
SQL>SELECT col1, col2, ... FROM table_name WHERE column multi_row_operator  
(SELECT column_name FROM table_name WHERE condition);
```

Here, **multi_row_operator** is one of:

- IN → equal to **any** value in the list
- ANY (or SOME) → compare with **at least one** value in the list
- ALL → compare with **all** values in the list

◆ Examples: Consider we have 'employee' table as-

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT
1	VIJAY	04-AUG-02	10000	HR
2	KIRAN	05-SEP-99	15000	IT
3	RAJESH	25-JUL-95	25000	SALES
4	SHANTA	12-MAY-05	18000	HR
5	AMIR	05-FEB-97	35000	HR

1. Using IN (equal to **any** value in the list)

Find employees who work in the **same department** as employee 'SHANTA'.

```
SQL> SELECT * FROM employee WHERE dept IN(SELECT dept FROM employee WHERE emp_name='SHANTA');
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT
5	AMIR	05-FEB-97	35000	HR
4	SHANTA	12-MAY-05	18000	HR
1	VIJAY	04-AUG-02	10000	HR

— Subquery may return multiple departments because emp_name 'SHANTA' may be more in table.

2. Using ANY (compare with **at least one** value in the list)

Find employees who earn **more than the lowest salary** in dept 'HR'.

```
SQL> select * from employee where salary > ANY(SELECT salary FROM employee WHERE dept='HR');
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT
2	KIRAN	05-SEP-99	15000	IT
3	RAJESH	25-JUL-95	25000	SALES
4	SHANTA	12-MAY-05	18000	HR
5	AMIR	05-FEB-97	35000	HR

— Equivalent to: greater than **minimum salary** of dept 'HR'

3. Using ALL (compare with **all** values in the list)

Find employees who earn **more than the highest salary** in department 'IT'.

```
SQL> select * from employee where salary > ALL(SELECT salary FROM employee WHERE dept='IT');
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT
3	RAJESH	25-JUL-95	25000	SALES
4	SHANTA	12-MAY-05	18000	HR
5	AMIR	05-FEB-97	35000	HR

— Equivalent to: greater than **maximum salary** of dept 'IT'.

4. Using NOT IN

Find employees who are **not in any department** where employee 'RAJESH' works.

```
SQL> select * from employee where dept NOT IN (SELECT dept FROM employee WHERE emp_name='RAJESH');
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT
1	VIJAY	04-AUG-02	10000	HR
2	KIRAN	05-SEP-99	15000	IT
4	SHANTA	12-MAY-05	18000	HR
5	AMIR	05-FEB-97	35000	HR

◆ Key Points OF Multi-row subquery-

1. Multi-row subqueries return **multiple rows but one column**.
2. Must use **multi-row operators** (IN, ANY, ALL, NOT IN etc).
3. Very useful when filtering based on **lists of values** returned from another table.

A.3) Multi-column subquery: (Tuple subquery)

- A **subquery that returns multiple columns** (also called a **tuple subquery**) is called 'Multi-column subquery'
- This subquery used when you need to **compare multiple columns (pairs/tuples)** of data together.
- In this subquery, outer query compares its own columns with the set of columns returned by the subquery.
- It is useful when you need to match **combinations of values** (not just a single column) between two tables or between a table and a subquery.

◆ Syntax

```
SQL> SELECT col1, col2, ... FROM table1 WHERE (colA, colB, ...) IN
(SELECT colX, colY, ... FROM table2 WHERE condition);
```

Here, (colA, colB) is compared **pair by pair** with (colX, colY) returned by the subquery.

Examples: Consider we have 'employee' table as-

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT	DEPT_ID	JOB_ID
1	VIJAY	04-AUG-02	10000	HR	d101	hr101
2	KIRAN	05-SEP-99	15000	IT	d102	it101
3	RAJESH	25-JUL-95	25000	SALES	d103	sales101
4	SHANTA	12-MAY-05	18000	HR	d101	hr101
5	AMIR	05-FEB-97	35000	HR	d101	hr102

1. Matching Department & Job

Find employees whose (dept_id, job_id) combination matches the (dept_id, job_id) of employee 'VIJAY'. (i.e. **Employees with Same Job & Department as VIJAY**)

```
SQL> SELECT * FROM employee WHERE (dept_id,job_id) IN (SELECT dept_id,job_id FROM employee WHERE emp_name='VIJAY');
```

EMP_ID	EMP_NAME	DOJ	SALARY	DEPT	DEPT_ID	JOB_ID
4	SHANTA	12-MAY-05	18000	HR	d101	hr101
1	VIJAY	04-AUG-02	10000	HR	d101	hr101

— The subquery returns **two columns** (dept_id, job_id) of VIJAY

The outer query compares **both values together**.

2. Multiple Column Match Across Different Tables:

Consider, we have two different table viz ‘orders’ and ‘sales’ as

```
SQL> select * from orders;
```

ORDER_ID	CUSTOMER_ID	CUST_NAME	PRODUCT_ID
101	1	RAMESH	501
102	2	SURESH	502
103	3	MEENA	503

```
SQL> select * from sales;
```

CUSTOMER_ID	PRODUCT_ID
1	501
2	502

Find **orders** where (**customer_id, product_id**) matches the records in a **sales** table.

```
SQL> SELECT order_id, customer_id, cust_name,product_id FROM orders WHERE (customer_id, product_id) IN (SELECT customer_id, product_id FROM sales);
```

ORDER_ID	CUSTOMER_ID	CUST_NAME	PRODUCT_ID
101	1	RAMESH	501
102	2	SURESH	502

◆ Key Points of multi-column subquery-

1. Multi-column subqueries return **multiple columns**.
2. Used when comparison must be done on **combinations of values** rather than a single column.
3. Works with IN, NOT IN (and sometimes EXISTS).
4. If you mistakenly use a single-column comparison operator (=), you will get an error.

A.4) Scaler subquery:

- A subquery that **returns exactly one value (one row and one column)** is called ‘**scalar subquery**’.
- Oracle treats this returned value just like a literal (constant) and allows it to be used anywhere in outer query.
- This subquery **must return only one value**; otherwise, Oracle throws an error (ORA-01427: single-row subquery returns more than one row).
- In short:

Scalar Subquery = Subquery that gives a single value which behaves like a constant in your SQL.

Syntax

```
SQL>SELECT column1, column2, ...FROM table1WHERE columnX = (SELECT single_column
FROM table2 WHERE condition );
```

Example: Consider we have 'employee' table as follow-

```
SQL> SELECT * FROM EMPLOYEE;
```

EMP_ID	EMP_NAME	DOJ	SALARY
1	VIJAY	04-AUG-02	10000
2	KIRAN	05-SEP-99	15000
3	RAJESH	25-JUL-95	25000
4	SHANTA	12-MAY-05	18000

Example 1 (Simple)

Find employees whose salary is **greater than salary of 'KIRAN'** in the table.

```
SQL> SELECT * FROM employee WHERE salary > (SELECT salary FROM employee WHERE emp_name='KIRAN');
```

EMP_ID	EMP_NAME	DOJ	SALARY
3	RAJESH	25-JUL-95	25000
4	SHANTA	12-MAY-05	18000

Here:

- Inner query → returns exact **single value** (salary of KIRAN).
- Outer query → compares salary against that one value.

Example 2 (Column in SELECT list)

You can even use a scalar subquery **inside the SELECT clause**.

Find all employees whose salary is less than **average salary of all employees**:

```
SQL> SELECT * FROM employee WHERE salary < (SELECT AVG(salary) FROM employee);
```

EMP_ID	EMP_NAME	DOJ	SALARY
1	VIJAY	04-AUG-02	10000
2	KIRAN	05-SEP-99	15000
4	SHANTA	12-MAY-05	18000

Here:

- Subquery returns **one value** (average salary).
- This value is compared with every row and produce the result.

Key Points About Scalar Subqueries

- Always return **one row, one column**.
 - Can be used in: WHERE clause, SELECT clause, HAVING clause, ORDER BY clause
- If it returns more than one row → Oracle error.

Note:

1) Main difference between *Single-Row Subquery* and *Scalar Subquery*

Aspect	Single-Row Subquery	Scalar Subquery
Definition	A subquery that returns exactly one row (but it may contain one or multiple columns).	A subquery that returns exactly one row and one column (i.e., a single value).
Return Type	One row (can be multiple columns).	One value (one row + one column).

2) Main difference between *Single-Column Subquery* and *Multi-Column Subquery*

Aspect	Single-Column Subquery	Multi-Column Subquery
Definition	A subquery that returns only one column .	A subquery that returns two or more columns .
Comparison	Compared with a single column of the outer query.	Compared with multiple columns of the outer query as a row pair .
Operators Used	=, <, >, IN, ANY, ALL	(col1, col2) IN (SELECT colA, colB ...)
Use Case	When you need to match/filter based on one field (e.g., salary, dept_id).	When you need to match/filter based on a combination of fields (e.g., salary + dept, dept + job_id).
Example Query	SQL> SELECT emp_name FROM employee WHERE dept = (SELECT dept FROM employee WHERE emp_name='VIJAY');	SQL>SELECT emp_name FROM employee WHERE (dept, job_id) IN (SELECT dept, job_id FROM employee WHERE emp_name='VIJAY');
Result	Employees in the same dept as VIJAY.	Employees in the same dept and job_id as VIJAY.

B) Based on Placement:

“Based on Placement” **subquery** means **where the subquery is written inside the SQL statement**

B.1) Subquery in WHERE clause:

- A **subquery** inside the WHERE clause is used to filter rows based on the result of another query.
- It allows you to compare a column in the main query with the output of the subquery.

General Syntax:

```
SQL> SELECT collist FROM table1 WHERE column_name operator ( SELECT Column_name FROM table2 WHERE condition );
```

Here,

- The **operator** can be =, <, >, IN, ANY, ALL, etc.

Example- (By considering above ‘employee’ table)

Find employees whose salary is **greater than the average salary** of all employees.

```
SQL> SELECT *FROM employee WHERE salary > ( SELECT AVG(salary) FROM employee);
```

EMP_ID	EMP_NAME	DOJ	SALARY
3	RAJESH	25-JUL-95	25000

Explanation:

- Inner query returns one value (average salary).
- Outer query selects employees with salary above that value.

B.2) Subquery in FROM clause:

- A **subquery in the FROM clause** is also called an **inline view**.
- It is used when you want to create a **temporary result set** (like a virtual table)

General Syntax

```
SQL>SELECT column_list FROM ( SELECT column1, column2, ...FROM table_name WHERE condition );
```

Note:

- The outer query treats the subquery result like a **virtual table**.

Example: (By considering above ‘employee’ table)

Find employees who earn more than 15000

SQL> SELECT * FROM (SELECT * FROM employee WHERE salary > 15000) ;			
EMP_ID	EMP_NAME	DOJ	SALARY
3	RAJESH	25-JUL-95	25000
4	SHANTA	12-MAY-05	18000

Explanation:

- Inner query (FROM) finds the salary > 15000
- Outer query filters employees whose salary is above 15000

B.3) Subquery in SELECT clause:

In Oracle, a **subquery in the SELECT clause** is also called a **scalar subquery**.

- It is a subquery that returns a **single value** (one row, one column).
- That value can then be displayed as a column in the outer query.

Syntax

```
SQL>SELECT column1, column2, ..., (SELECT single_column FROM table2 WHERE condition ) FROM table1;
```

Example: By considering above 'employee' table-

Show employee salary along with average salary

SQL> SELECT emp_id,emp_name,salary, (SELECT AVG(salary) FROM employee) AS avg_sal FROM employee;			
EMP_ID	EMP_NAME	SALARY	AVG_SAL
1	VIJAY	10000	17000
2	KIRAN	15000	17000
3	RAJESH	25000	17000
4	SHANTA	18000	17000

Here, the subquery (SELECT AVG(salary) FROM employee) returns one value (the overall average salary), and that value is compared with every row to get the output.

C) Based on Correlation:

“Based on Correlation” **subquery** means whether the **subquery depends on the outer query or not**.

C.1) Correlated subquery:

- A **correlated subquery** is a subquery that **depends on the outer query** for its result.
- **Correlated subquery** executes **once for every row** returned by **the outer query**.
- Inside correlated subquery, we use **reference to a column of the outer query**.
- This correlated subquery is useful for row-by-row comparisons.

General Syntax:

```
SQL>SELECT colList FROM mainTable m WHERE condition (SELECT colList FROM otherTable o WHERE o.col = m.col);
```

Here,

‘m’ is reference to mainTable

‘o’ is reference to otherTable

Also, this subquery depends on outer query for that condition is mentioned **o.col=m.col**

Example: Consider we have two tables ‘customer’ and ‘orders’ as follow-

```
SQL> select * from customer;
```

CUST_ID	CUST_NAME	CITY
1	Amit Sharma	Delhi
2	Priya Mehta	Mumbai
3	Ravi Kumar	Bengaluru
4	Neha Singh	Kolkata
5	Arjun Reddy	Hyderabad

```
SQL> select * from orders;
```

ORDER_ID	CUST_ID	ORDER_AMOUNT
101	1	4500
102	1	7000
103	2	3200
104	3	8500
105	4	2100

Example- 1: Find customers whose maximum order amount is greater than 6000

```
SQL> SELECT * FROM customer x WHERE 6000 < (SELECT MAX(order_amount) FROM orders y WHERE y.cust_id = x.cust_id);
```

CUST_ID	CUST_NAME	CITY
1	Amit Sharma	Delhi
3	Ravi Kumar	Bengaluru

Explanation:

- Outer query → goes row by row in **CUSTOMER**.
- Inner query → finds the **maximum order amount** for that specific customer (o.cust_id = c.cust_id).
- Condition → returns only those customers whose **max order > 6000**.

Example- 2: Find customers whose maximum order amount is less than 5000

(Without creating reference to tables. Here, we use table name as reference)

```
SQL> SELECT * FROM customer WHERE 5000 > (SELECT MAX(order_amount) FROM orders WHERE orders.cust_id = customer.cust_id);
```

CUST_ID	CUST_NAME	CITY
2	Priya Mehta	Mumbai
4	Neha Singh	Kolkata

Explanation:

- Outer query → goes row by row in **CUSTOMER**.
- Inner query → finds the **maximum order amount** for that specific customer (**orders.cust_id = customer.cust_id**).
- Condition → returns only those customers whose **max order < 5000**.

C.2) Non-correlated subquery: (Independent subquery)

- A **Non-Correlated Subquery** (also called **Independent Subquery**) is a subquery whose execution **does not depend on the outer query**.
- It can be executed **independently** to produce a result, and this result takes as input by outer query.
- This non-correlated subquery also runs **once independently**.
- It is typically used in WHERE, SELECT, or FROM clauses.
- It **does not reference any column from the outer query**.

General Syntax:

```
SQL>SELECT col1, col2, ... FROM table1 WHERE columnX operator (SELECT columnY  
FROM table2 WHERE condition);
```

Example: Consider we have two tables 'customer' and 'orders' as above.

1: Find customers who made orders greater than 6000

```
SQL> SELECT * FROM CUSTOMER WHERE cust_id IN (SELECT cust_id FROM ORDERS WHERE order_amount > 6000);
```

CUST_ID	CUST_NAME	CITY
1	Amit Sharma	Delhi
3	Ravi Kumar	Bengaluru

Explanation:

1. The **subquery** (SELECT cust_id FROM ORDERS WHERE order_amount > 6000) runs **first** independently. **Output:** cust_id = 2, 4
2. The **outer query** then selects * from CUSTOMER for these cust_id's.
3. This is **non-correlated** because the subquery does **not depend on any value from the outer query**.

Handling data retrieval with **EXISTS** and **NOT EXISTS** operators in subquery-

- **EXISTS:** Returns **TRUE** if the subquery returns one or more rows.
- **NOT EXISTS:** Returns **TRUE** if the subquery returns no rows.
- **Important:** EXISTS does **not care about the actual data** returned by the subquery. It only checks whether rows exist.

Syntax for EXISTS:

```
SQL>SELECT columns FROM table1 outer WHERE EXISTS  
( SELECT 1 FROM table2 inner WHERE inner.column = outer.column);
```

Syntax for NOT EXISTS:

```
SQL>SELECT columns FROM table1 outer WHERE NOT EXISTS  
(SELECT 1 FROM table2 inner WHERE inner.column = outer.column );
```

Here, **SELECT 1** is common, but you could select anything—it's ignored by Oracle.

Example- Consider we have two tables 'customer' and 'orders' as above.

Example 1: Use of EXISTS

Find customers who have placed at least one order.

```
SQL> SELECT * FROM customer x WHERE EXISTS ( SELECT 1 FROM orders y WHERE y.cust_id = x.cust_id);
```

CUST_ID	CUST_NAME	CITY
1	Amit Sharma	Delhi
2	Priya Mehta	Mumbai
3	Ravi Kumar	Bengaluru
4	Neha Singh	Kolkata

Explanation:

1. The subquery checks for **each customer** if a row exists in ORDERS with the same cust_id.
2. If **at least one order exists**, EXISTS returns TRUE.

Example 2: Use of NOT EXISTS

Find customers who have **never placed an order**.

```
SQL> SELECT * FROM customer x WHERE NOT EXISTS (SELECT 1 FROM ORDERS y WHERE y.cust_id = x.cust_id);
```

CUST_ID	CUST_NAME	CITY
5	Arjun Reddy	Hyderabad

Explanation:

- The subquery checks if there is any order for each customer.
- NOT EXISTS returns **TRUE** if **no order exists** for that customer.

Joins:

- A JOIN in Oracle is used to combine (join) rows i.e. records i.e. data from two or more tables based on a related column between them.
- Join helps to fetch meaningful data which is spread across multiple tables.

Advantages or Characteristics or Properties or Importance of Joins:

1. **Combine Related Data** → Using joins we can fetch the data from multiple tables & combine it in single query.
2. **Avoid Redundancy** → Data duplication is avoided by using joins.
3. **Efficient Querying** → Joins are Faster and more powerful than running multiple separate queries i.e. it is efficient in querying.
4. **Flexibility** → Different join types (INNER, OUTER, SELF, CROSS etc.) helps in different business needs hence it is flexible.
5. **Supports Normalization** → Joins works well with normalized databases (data split across tables).
6. **Better Data Analysis** → Joins fulfils the requirements in complex reporting and business area for better data analysis.
7. **Data Integrity** → Joins ensures relationships between tables are utilized properly.

Types of Joins:

Following are the different types of joins-

- 1) Inner Join
- 2) Outer Join
- 3) Self-Join
- 4) Cartesian Join (Cross-Join)

Let's see these types in details-

1) Inner Join-

- It is type of join where it returns only the rows (i.e. records) that have matching values in both tables
- That is inner join excludes non-matching rows.

Syntax for Inner join-

```
SQL>SELECT colList FROM table1 INNER JOIN table2 ON table1.column_name = table2.column_name;
```

Explanation

- **INNER JOIN** keyword specifies the join type.
- **ON** defines the join condition (the relationship between the two tables).

Here, only those rows are returned that satisfy the condition.

Example- Consider we have 'customer' and 'orders' table as follow-

```
SQL> select * from customer;
```

CUST_ID	CUST_NAME	CITY
1	Amit	Pune
2	Ravi	Mumbai
3	Sita	Delhi

```
SQL> select * from orders;
```

ORDER_ID	CUST_ID	ORDER_AMOUNT
101	1	500
102	1	150
103	2	700

1) Applying Inner join onto above tables to retrieve **only those customers who have placed at least one order**.

```
SQL> SELECT * from customer INNER JOIN orders ON orders.cust_id=customer.cust_id;
```

CUST_ID	CUST_NAME	CITY	ORDER_ID	CUST_ID	ORDER_AMOUNT
1	Amit	Pune	101	1	500
1	Amit	Pune	102	1	150
2	Ravi	Mumbai	103	2	700

2) Outer Join:

- An **Outer Join** returns **all rows from one table**, and the **matching rows from the other table**.
- If no match is found, **NULL values** are returned for missing columns.
- Outer join helps when you want to see unmatched records also (not just the common ones).

Types of Outer Joins:

2.1) LEFT OUTER JOIN → Returns **all rows from left table**, and **matched rows from right table**.

2.2) RIGHT OUTER JOIN → Returns **all rows from right table**, and **matched rows from left table**.

2.3) FULL OUTER JOIN → Returns all rows from **both tables** whether they are matched or unmatched.

Let's see these types in details-

2.1) LEFT OUTER JOIN:

A **LEFT OUTER JOIN** returns:

- **All rows from the left table** (even if no match exists in the right table).
- Matching rows from the right table.
- If there's **no match**, Oracle fills the **right table columns** with **NULL**.

Syntax for LEFT OUTER JOIN:

```
SQL> SELECT colList FROM table1 LEFT OUTER JOIN table2 ON table1.column = table2.column;
```

Here, table1 → left table (all rows returned).

table2 → right table (only matched rows returned, unmatched become NULL).

☞ The keyword **OUTER** is optional, you can simply write **LEFT JOIN**.

Example- Consider we have 'customer' and 'orders' table as above.

1) By applying LEFT OUTER JOIN, we get output as-

```
SQL> SELECT x.cust_id,x.cust_name,x.city,y.order_id,y.order_amount FROM customer x LEFT OUTER JOIN orders y ON x.cust_id=y.cust_id;
```

CUST_ID	CUST_NAME	CITY	ORDER_ID	ORDER_AMOUNT
1	Amit	Pune	101	500
1	Amit	Pune	102	150
2	Ravi	Mumbai	103	700
3	Sita	Delhi		

☞ Explanation:

Amit (CUST_ID = 1) has two orders, so he appears twice.

Ravi (CUST_ID = 2) has one order.

Sita (CUST_ID = 3) has no order, but still appears in the result with **NULLs** for order columns (because of the **LEFT OUTER JOIN**).

2.2) RIGHT OUTER JOIN:

- A **RIGHT OUTER JOIN** returns:
 - **All rows from the right table**. (even if no match exists in the left table).
 - It also returns the **matching rows from the left table**.
 - If **no match exists**, Oracle fills the **left table's columns** with **NULLs**.
- It is basically the **reverse of LEFT OUTER JOIN**.

Syntax for RIGHT OUTER JOIN:

```
SQL>SELECT colList FROM table1 RIGHT OUTER JOIN table2 ON table1.column = table2.column;
```

Here,

table1 → left table

table2 → right table (all rows will be included)

Example- Consider we have 'customer' and 'orders' table as follow-

```
SQL> select * from customer;
```

CUST_ID	CUST_NAME	CITY
1	Amit	Pune
2	Ravi	Mumbai
3	Sita	Delhi

```
SQL> select * from orders;
```

ORDER_ID	CUST_ID	ORDER_AMOUNT
101	1	500
102	1	150
103	2	700
104	5	250

1) By applying RIGHT OUTER JOIN, we get output as-

```
SQL> SELECT x.cust_id,x.cust_name,x.city,y.order_id,y.order_amount FROM customer x RIGHT OUTER JOIN orders y ON x.cust_id=y.cust_id;
```

CUST_ID	CUST_NAME	CITY	ORDER_ID	ORDER_AMOUNT
1	Amit	Pune	102	150
1	Amit	Pune	101	500
2	Ravi	Mumbai	103	700
			104	250

🔗 Explanation:

- Customers **Amit** and **Ravi** appear with their orders.
- Order 104 belongs to a customer ID 5 (not in CUSTOMER table), so the customer details show NULL.
- Customer **Sita** (3) doesn't appear, because RIGHT JOIN prioritizes the orders table first.

2.3) FULL OUTER JOIN:

- A **FULL OUTER JOIN** returns:
 - All rows from the left table.
 - All rows from the right table.
 - If there is **no match between the tables**, it fills with **NULL** for the missing side.
- FULL outer join is combination of **LEFT OUTER JOIN** and **RIGHT OUTER JOIN**.

Syntax for FULL OUTER JOIN:

```
SQL>SELECT columns FROM table1 t1 FULL OUTER JOIN table2 t2 ON t1.column_name = t2.column_name;
```

Example- Consider we have 'customer' and 'orders' table as above.

1) By applying FULL OUTER JOIN, we get output as-

```
SQL> SELECT x.cust_id,x.cust_name,x.city,y.order_id,y.order_amount FROM customer x FULL OUTER JOIN orders y ON x.cust_id=y.cust_id;
```

CUST_ID	CUST_NAME	CITY	ORDER_ID	ORDER_AMOUNT
1	Amit	Pune	101	500
1	Amit	Pune	102	150
2	Ravi	Mumbai	103	700
3	Sita	Delhi		
			104	250

Explanation:

- Customer Amit (ID 1) has 2 orders → shows 2 rows.
- Customer Ravi (ID 2) has 1 order → 1 row.
- Customer Sita (ID 3) has no order → shows with **NULL order columns**.
- Order with CUST_ID = 5 has no matching customer → shows with **NULL customer columns**.

3) SELF JOIN:

- A SELF JOIN is a **join in which a table is joined with itself**.
- It is useful when rows of the same table need to be compared, for example:
 - Finding customers who live in the same city within same table.
 - Finding employees who are managers of other employees within same table.

Syntax for SELF JOIN-

```
SQL>SELECT a.col, b.col FROM table a, table b WHERE condition;
```

Here,

a and **b** are aliases OR references for the same table.

Example- Consider we have 'employee' table as follow-

```
SQL> select * from employee;
```

EMP_ID	EMP_NAME	MANAGER_ID
1	Amit	
2	Ravi	1
3	Sita	1
4	John	2

☞ **Note:** Here, MANAGER_ID refers to EMP_ID of the manager.

1) Applying SELF JOIN on above table we get output as-

```
SQL> SELECT a.emp_id,a.emp_name AS MANAGER,b.emp_name AS EMP FROM employee a,employee b WHERE a.emp_id=b.manager_id;
```

EMP_ID	MANAGER	EMP
1	Amit	Ravi
1	Amit	Sita
2	Ravi	John

Explanation:

- Ravi and Sita have Amit as manager.
- John has Ravi as manager.
- Amit has no manager (NULL).

4) Cartesian Join (Cross-Join):

- A Cartesian Join (also called Cross Join) happens **when each row of one table is joined with every row of another table**.
- It usually happens when you don't specify a proper join condition.
- The number of resulting rows after Cartesian join = **(rows in table1 × rows in table2)**

Syntax:

```
SQL>SELECT x.col,y.col... FROM table1 x CROSS JOIN table2 y;
```

Example- Suppose we have two tables as 'dept' and 'emp' as follow-

```
SQL> SELECT * FROM dept;

DEPT_ID DEPT_NAME
-----
10 HR
20 IT

SQL> SELECT * FROM emp;

EMP_ID EMP_NAME DEPT_ID
-----
1 Amit 10
2 Ravi 20
3 Sita 10
```

Applying CROSS JOIN onto above table we get-

```
SQL> SELECT e.emp_id,e.emp_name,e.dept_id,d.dept_name FROM emp e CROSS JOIN dept d;

EMP_ID EMP_NAME DEPT_ID DEPT_NAME
-----
1 Amit 10 HR
2 Ravi 20 HR
3 Sita 10 HR
1 Amit 10 IT
2 Ravi 20 IT
3 Sita 10 IT
```

Lossless Join:

- Lossless join is the concept in **Database Normalization**, it means that when a relation (table) is decomposed into two or more smaller relations (tables), we can join them back without losing any information.
- Lossless join ensures that if a table is broken into two or more tables, you can reconstruct the original table by joining those decomposed tables without losing any data.
- This property is important for database design to avoid redundancy and inconsistency.

Syntax for Lossless join:

When you decompose a table R into two tables R1 and R2,

```
SQL>SELECT * FROM R1 NATURAL JOIN R2;
```

If the result gives back all rows of the original relation R, then the decomposition is **lossless**.

Example-

1. Original Table: Employee

EMP_ID	EMP_NAME	DEPT_ID	DEPT_NAME
1	Amit	10	HR
2	Ravi	20	IT
3	Sita	10	HR

2. Decompose into Two Tables

Dept Table

```
SQL>CREATE TABLE dept (  
    dept_id    NUMBER(5) PRIMARY KEY,  
    dept_name  VARCHAR2(5)  
);  
  
SQL>INSERT INTO dept VALUES (10, 'HR');  
SQL>INSERT INTO dept VALUES (20, 'IT');
```

Emp Table

```
SQL>CREATE TABLE emp (  
    emp_id     NUMBER(5) PRIMARY KEY,  
    emp_name   VARCHAR2(10),  
    dept_id    NUMBER(5)  
);  
  
SQL>INSERT INTO emp VALUES (1, 'Amit', 10);  
SQL>INSERT INTO emp VALUES (2, 'Ravi', 20);  
SQL>INSERT INTO emp VALUES (3, 'Sita', 10);
```

3. Reconstruct (Join them back)

```
SQL>SELECT emp_id,emp_name,dept_id,dept_name FROM emp NATURAL JOIN dept;
```

EMP_ID	EMP_NAME	DEPT_ID	DEPT_NAME
1	Amit	10	HR
2	Ravi	20	IT
3	Sita	10	HR

This is exactly the **original table**, so the decomposition is **lossless join**.

SEQUENCE:

- A SEQUENCE is a database object which is used to **generate unique numbers automatically**.
- SEQUENCE basically used for primary keys field in a table (like student roll number, employee IDs, invoice numbers, order IDs, etc.) where these fields increments sequentially.
- SEQUENCE saves you from manually inserting unique values in table.
- In short a **SEQUENCE in Oracle is like an auto-number generator**.

Why is SEQUENCE important?

- 1) **Automatic primary keys** → No need to worry about duplicate IDs while inserting rows.
- 2) **Performance** → Sequences are very fast, optimized by Oracle.
- 3) **Flexibility** → You can control start value, increment, min/max values, and whether numbers can cycle (start over).
- 4) **Concurrency** → Even if many users insert rows at the same time, sequence guarantees unique numbers.
- 5) **Ensures uniqueness** → Each call to NEXTVAL generates a **unique number** (no duplicates).

Let's see some **Oracle SEQUENCE commands-**

1. Create a Sequence:

➤ To create new sequence, CREATE SEQUENCE command is used.

```
SQL> CREATE SEQUENCE sequenceName
      START WITH n
      INCREMENT BY m
      MINVALUE x
      MAXVALUE y
      CYCLE | NOCYCLE
      CACHE n | NOCACHE;
```

Example:

```
SQL> CREATE SEQUENCE emp_seq
      START WITH 1
      INCREMENT BY 1
      MINVALUE 1
      MAXVALUE 1000
      NOCYCLE
      CACHE 20;
```

Here, SEQUENCE 'emp_seq' is created having following properties-

- Starts at **1**, increases by **1** each time.
- Minimum = 1, Maximum = 1000.
- **NOCYCLE** → won't restart after reaching max.
- **CACHE 20** → Oracle pre-allocates 20 numbers in memory (for speed).

2. Retrieve Data from a Sequence:

To retrieve data from sequence, two keywords are used which are as follow-

- **NEXTVAL** → gets the next number in the sequence.
- **CURRVAL** → gets the current value of the sequence.
- **NOTE:** We can use **CURRVAL** after **NEXTVAL** has been used once in the session.

Example: We use NEXTVAL and CURRVAL on above created 'emp_seq' sequence as follow-

```
SQL> select emp_seq.nextval from dual;

NEXTVAL
-----
      1

SQL> select emp_seq.nextval from dual;

NEXTVAL
-----
      2

SQL> select emp_seq.nextval from dual;

NEXTVAL
-----
      3

SQL> select emp_seq.currval from dual;

CURRVAL
-----
      3
```

3. Modify a Sequence:

- We can modify sequence using `ALTER SEQUENCE` command.
- `ALTER SEQUENCE` command changes the properties of created sequence.
- Note that, we **cannot** alter or modify the **START WITH** property of sequence.
- **Syntax-**

```
SQL>ALTER SEQUENCE sequence_name
      INCREMENT BY n
      MAXVALUE x
      MINVALUE y
      CYCLE | NOCYCLE
      CACHE n | NOCACHE;
```

Example-

```
SQL> alter sequence emp_seq increment by 5 maxvalue 5000;

Sequence altered.

SQL> select emp_seq.CURRVAL FROM dual;

      CURRVAL
-----
           3

SQL> select emp_seq.NEXTVAL FROM dual;

      NEXTVAL
-----
           8
```

- Here, the sequence 'emp_seq' will increase by **5** instead of **1**.

4. Drop a Sequence:

- To drop created sequence, `DROP SEQUENCE` command is used.

Syntax:

```
SQL> DROP SEQUENCE sequence_name;
```

Example:

```
SQL> DROP SEQUENCE emp_seq;
```

This completely removes the sequence from the database.

Following example shows actual use of SEQUENCE while inserting record in table-

```
SQL> CREATE TABLE stud(rollno number PRIMARY KEY,name varchar2(10),per number);
Table created.

SQL> CREATE SEQUENCE rollseq START WITH 3201 INCREMENT BY 1 MINVALUE 3201 MAXVALUE 3301 NOCYCLE NOCACHE;
Sequence created.

SQL> INSERT INTO stud VALUES(rollseq.NEXTVAL,'JIBRAIL',86.52);
1 row created.

SQL> SELECT * FROM stud;

  ROLLNO NAME      PER
-----
  3201 JIBRAIL      86.52

SQL> INSERT INTO stud VALUES(rollseq.NEXTVAL,'KISHOR',58.52);
1 row created.

SQL> INSERT INTO stud VALUES(rollseq.NEXTVAL,'ROHIT',78.45);
1 row created.

SQL> SELECT * FROM stud;

  ROLLNO NAME      PER
-----
  3201 JIBRAIL      86.52
  3202 KISHOR        58.52
  3203 ROHIT         78.45
```

SYNONYM:

- In Oracle Database, a **synonym** is simply an **alias (alternative name)** for a database object.
- SYNONYM allows you to **give your own name to the database object** (like a table, view, sequence, procedure, or another synonym) which is **often shorter or more meaningful**.

Why to use Synonyms?

1. To provide a **shorter name** for long object names.
2. To **hide schema details** (users don't need to know the owner's schema).
3. To make it easier for **applications** to access objects **without schema prefix**.
4. To allow **location transparency** when **accessing remote objects**.

Types of Synonyms:

1. **Private synonym** – Private synonym is **accessible only within the user's schema** where it is created. (It is default synonym)
2. **Public synonym** – Public synonym is **accessible by all users of database**. (It is **not default** synonym; **PUBLIC** keyword is used to make synonym public)

Syntax:

1)Creating a synonym:

```
SQL>CREATE PUBLIC SYNONYM synonym_name FOR object_name;
```

Here, **PUBLIC** is keyword is **optional**. It is used to make synonym as public.
If we **do not use PUBLIC**, then by default synonym is **PRIVATE**.

2)Dropping a synonym:

```
SQL>DROP PUBLIC SYNONYM synonym_name;
```

Here, **PUBLIC** keyword is used to drop **PUBLIC** synonym.
To drop **PRIVATE** synonym, **no need to specify any keyword**.

Example 1) **PRIVATE** synonym: Example shows creating and using the **PRIVATE** synonym.

```
SQL> create table SangolaCollegeEmployee(eid number,ename varchar2(10));
Table created.
SQL> CREATE SYNONYM sglemp for SangolaCollegeEmployee;
Synonym created.
SQL> insert into sglemp values(1,'JIBRAIL');
1 row created.
SQL> insert into sglemp values(2,'ROHIT');
1 row created.
SQL> SELECT * from sglemp;

  EID ENAME
-----
    1 JIBRAIL
    2 ROHIT
```

Example 2) **PUBLIC** synonym: Example shows creating and using the **PUBLIC** synonym.

```
SQL> create table UniversityEmployee(eid number,ename varchar2(10));
Table created.
SQL> CREATE PUBLIC SYNONYM uniemp for UniversityEmployee;
Synonym created.
SQL> insert into uniemp values(101,'PIYUSH');
1 row created.
SQL> insert into uniemp values(102,'GANESH');
1 row created.
SQL> SELECT * FROM uniemp;

  EID ENAME
-----
  101 PIYUSH
  102 GANESH
```

Note:

PUBLIC synonym is accessible by all users of database.

PRIVATE synonym is only accessible within user where it is created.

(Show practically for student- OPEN two RUN SQL command Line one for SYSTEM & one for other created user (is should have CONNCT privileges))

Hint: SQL>Grant SELECT ON synoName TO userName;

For both PUBLIC and PRIVATE synonym and check the result on other user terminal)

Index-

In Oracle, an **INDEX** is a database object basically **used for faster access of rows i.e. records from a table** based on values of one or more columns.

— We know that, every book has an index, it contains chapter name and its equivalent page number.

To search any chapter, first we check whether it is present in index or not. If chapter is present in index, then we directly switch to its page number (location), instead of scanning the whole book.

➤ Thus, **index makes searching fast.**

Advantages of Index:

1) Faster Data Retrieval:

Indexes provide a quick access path to rows (records) of a table.

2) Improves Query Performance

Indexes useful in queries with WHERE, JOIN, ORDER BY, and GROUP BY clauses.

Hence, Sorting and grouping become faster.

3) Efficient Access Using Rowids:

Index stores **ROWID** of each row of a table then Oracle can directly jump to the exact location of data.

4) Speeds up Joins

Indexes also speed up join columns (like employee_id, department_id) that improve performance of multi-table joins.

5) Helps Enforce Uniqueness:

Unique indexes are automatically created when you define **PRIMARY KEY** or **UNIQUE constraints**, ensuring no duplicate values.

6) Supports Function-Based Queries

Indexes support for Function-based queries on expressions or strings like UPPER(name) etc. that makes faster execution.

7) Better Sorting and Searching

Indexes help queries that use ORDER BY and range searches (BETWEEN, <, >) perform efficiently.

8) Improves Aggregation Performance

Queries with aggregates (SUM, COUNT, AVG) on indexed columns execute faster.

Types of Index:

In Oracle, there are several **types of indexes**, each type is useful depending on the data and query requirements. Which are explained as follow-

1. B-tree Index (Default)

- It is most common type of index that stores data in a balanced tree structure.
- This B-tree index is best for **high-cardinality columns** (i.e. column has many **distinct** values).
- B-tree index is ideal for **searching unique values**.

Syntax:

```
SQL>CREATE INDEX indexName ON tableName(ColName);
```

Example:

```
SQL> CREATE INDEX idx_name ON employees(name);
```

Here, we create an index 'idx_name' on the 'name' column of the 'employees' table

Now, we retrieve data using created index as-

```
SQL>SELECT * FROM employees WHERE name = 'MAHESH';
```

Here, Oracle uses the **B-tree index** (idx_name) to quickly find the ROWID of 'MAHESH' and fetch the row.

2. UNIQUE INDEX

- **UNIQUE INDEX** ensures all values in the indexed column are unique.
- If you try to insert a duplicate value in **unique index column** then oracle generates error.
- **Note that**, **UNIQUE INDEX** automatically created when you define a **PRIMARY KEY** or **UNIQUE constraint** for a column.

Syntax:

```
SQL>CREATE UNIQUE INDEX indexName ON tableName(ColName);
```

Example:

```
SQL> desc emp;
Name                               Null?    Type
-----
ID                                   CHAR(2)
ENAME                               VARCHAR2(20)
EMAIL                               VARCHAR2(20)

SQL> CREATE UNIQUE INDEX indx_unq ON emp(email);

Index created.

SQL> insert into emp values(1,'Ramesh Mane','ramesh121@gmail.com');

1 row created.

SQL> insert into emp values(2,'Ramesh Kale','ramesh121@gmail.com');
insert into emp values(2,'Ramesh Kale','ramesh121@gmail.com')
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.INDX_UNQ) violated
```

3. Composite Index (Concatenated Index)

- If **Index** is created on **two or more columns** of table, then it is called 'Composite Index'
- This **COMPOSITE INDEX** is useful when **filtering is applied on multiple columns**.

Syntax:

```
SQL>CREATE INDEX indexName ON tableName(Col1,Col2,...);
```

Example:

```
SQL> CREATE INDEX idx_cmp ON emp(eid,ename);
```

Here, we create a composite index '**idx_cmp**' on the '**eid**' and '**ename**' columns of the '**emp**' table
Now, we retrieve data using created index as-

```
SQL> SELECT * FROM emp WHERE eid= 101 AND ename= 'KIRAN';
```

-- Above query uses **both indexed** columns **eid** and **ename** for filtering. Hence, data will fetch fast.

4. Bitmap Index

- This index uses **bitmaps (string of bits, 0s and 1s)** for indexing instead of B-trees.
- This index is best for **low-cardinality columns** (few distinct values, like gender, status).

Syntax:

```
SQL> CREATE BITMAP INDEX indexName ON tableName(Column);
```

Example:

```
SQL> CREATE BITMAP INDEX idx_bit ON emp(gender);
```

Here, we create a bitmap index '**idx_bit**' on the '**gender**' column of the '**emp**' table.

Retrieving data with bitmap index-

```
SQL> SELECT COUNT(*) FROM emp WHERE gender = 'M';
```

-- Above query uses **indexed** column **gender** to fetch all Male employees from '**emp**' table.

5. Function-Based Index

- If index is **created on the result of a function or result of expression** then it is called 'Function-Based Index'.
- Function based index **helps to optimize queries where functions are used** in WHERE clauses.

Syntax:

```
SQL> CREATE INDEX indexName ON tableName(fun_Name(col1,col2));
```

Here, **fun_name()** is built in function or user defined function in oracle on to which we create index.

Example:

```
SQL> CREATE INDEX idx_fun ON emp(UPPER(ename));
```

Here, we create a function based index '**idx_fun**' on the '**UPPER**' function having column **ename**.

Retrieving data with function based index-

```
SQL> SELECT * FROM emp WHERE UPPER(ename) = 'JAY';
```

6. Descending Index

- Descending Index is **used to store data in descending order** in table depends on column.
- It is useful for **queries using ORDER BY ... DESC**.

Syntax:

```
SQL> CREATE INDEX indexName ON tableName(Col DESC);
```

Example:

```
SQL> CREATE INDEX idx_dec ON emp(salary DESC);
```

Here, we create a descending index '**idx_dec**' on column '**salary**' of table '**emp**'.

Retrieving data with descending index-

```
SQL> SELECT * FROM emp ORDER BY salary DESC;
```

Pseudo columns:

- In Oracle Database, a **pseudo column** is a special type of column **that appears to exist in a table but is not actually stored there**.
- Pseudo column is **automatically provided by Oracle** and **can be queried like a normal column**, but you **cannot insert, update, or delete values** in it.
- Pseudo column is **virtual column** provided by Oracle for special purposes.

Commonly Used Pseudo Columns in Oracle:

1. ROWID

- The ROWID pseudo column **represents the unique physical address of a row** in a table.
- Whenever we insert new row in table, oracle automatically assign ROWID for that row.

Example:

```
SQL> SELECT ROWID,emp_name from emp;

ROWID                EMP_NAME
-----
AAADVMAABAAAKVKAAA  Amit
AAADVMAABAAAKVKAAB  Ravi
AAADVMAABAAAKVKAAC  Sita
```

2. ROWNUM

- ROWNUM pseudo column **represents the sequence number of a row** present in a table.
- Basically ROWNUM is **used to limit rows** of table.

Example:

```
SQL> SELECT ROWNUM,emp_name FROM emp;

ROWNUM EMP_NAME
-----
1 Amit
2 Ravi
3 Sita
```

3. CURRVAL & NEXTVAL

- CURRVAL & NEXTVAL are pseudo columns **used with sequences**.
- NEXTVAL gives the **next value from a sequence**.
- CURRVAL gives the **current value of a sequence**.

Example:

```
SQL> SELECT emp_seq.NEXTVAL from dual;

NEXTVAL
-----
8

SQL> SELECT emp_seq.CURRVAL from dual;

CURRVAL
-----
8
```

4. LEVEL

- LEVEL is pseudo column **used in hierarchical queries (CONNECT BY)** indicates the level of a **node in a tree structure**.

Example:

```
SQL> SELECT * FROM emp;
```

EMP_ID	EMP_NAME	MGR_ID
1	Amit	
2	Ravi	1
3	Sita	1
4	John	2

```
SQL> SELECT emp_name, LEVEL FROM emp CONNECT BY PRIOR emp_id = mgr_id START WITH mgr_id IS NULL;
```

EMP_NAME	LEVEL
Amit	1
Ravi	2
John	3
Sita	2

Explanation-

1. Table assumption (emp)

- emp_id → Employee ID (unique for each employee).
- mgr_id → Manager ID (refers to another emp_id).
- emp_name → Employee's name.

This means:

- Each employee may have a manager (mgr_id).
- The **top boss** will have mgr_id = NULL. Here, 'Amit' is boss.

2. START WITH mgr_id IS NULL

- This defines the **root(s) of the hierarchy**.

Here, the hierarchy starts with employees who **don't have a manager** (the CEO/top-level bosses).

3. CONNECT BY PRIOR emp_id = mgr_id

- This defines the parent → child relationship.
- PRIOR emp_id = the parent's emp_id (from the row above in the hierarchy).
- mgr_id = the child's manager ID.

Views:

- In Oracle Database, a **view** is a **virtual table** based on the result of a SQL query.
- View **does not store data physically**, but it **provides a way to look at data from one or more tables in a customized manner**.
- A view is a **named SQL query** stored in the database. It **Acts like a table** but does **not store data**.
- **Use of VIEW:**
 - VIEW used for **simplifying complex queries**
 - VIEW provides a level of **security for data** by **restricting access to specific columns/rows**.
 - VIEW used for presenting **aggregated or joined data**.
 - VIEW **Hides** underlying **table structures**.

Syntax:

```
SQL> CREATE [OR REPLACE] [FORCE | NOFORCE] VIEW view_name AS
SELECT query [WITH CHECK OPTION] [WITH READ ONLY];
```

Here,

- **CREATE OR REPLACE:** If view **not exists** then it creates new view. If view already exists then it replaces existing view by new one.
- **FORCE:** Creates the view even if the **base tables don't exist** (use with caution).
- **NOFORCE:** Default. **Base tables must exist**.
- **WITH CHECK OPTION:** Ensures that **DML operations via the view do not violate**.
- **WITH READ ONLY:** Makes the view non-updatable (**no INSERT/UPDATE/DELETE**).

Types of View:

1. Relational View:

- A **Relational View** is simply a view **based on one or more relational tables** and that presents data from these tables in a specific format.
- The relational view basically, a **virtual table derived from base relational tables** using SQL queries.
- Relational views help simplify complex data structures by providing relational data.

Key points about Relational Views:

- This view **based on one or more relational tables**.
- This view **does not store data physically**, here **data is dynamically fetched when queried**.
- This view provides **security by restricting columns or rows**.
- This view can be **simple** (based on one table) or **complex** (based on multiple tables, joins, group by, etc).
- This view can be **updatable (allow insert/update/delete) or read-only** depending on the SQL used **WITH READ ONLY** etc.
- If view is created **WITH READ ONLY** then it will **not updatable**. But if view is created **without READ ONLY** then view **can be updatable**.
- If view is **updated** then its **respective table also affects** by new values.

Syntax for Creating a Relational View

```
SQL>CREATE [OR REPLACE] VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
[WHERE condition]
[GROUP BY ...]
[HAVING ...]
[ORDER BY ...];
```

Types of Relational view-

1.1) Simple Relational View-

Definition:

A view is **created on a single table without any joins, aggregates, or derived columns** is called 'Simple relational view'

Consider we have two tables 'dept' and 'emp' as follow-

```
SQL> SELECT * FROM dept;
```

DEPT_ID	DEPT_NAME	LOCATION
10	HR	Delhi
20	IT	Mumbai
30	Finance	Pune

```
SQL> SELECT * FROM emp;
```

EMP_ID	EMP_NAME	SALARY	DEPT_ID
101	Amit Sharma	40000	10
102	Neha Gupta	50000	20
103	Ravi Kumar	45000	20
104	Sunita Mehta	60000	30

Example: Simple Relational View (Single Table)

We create a simple relational view to show **employees** who works in **department number 20**.

```
SQL> CREATE VIEW simple_view AS SELECT * FROM emp WHERE dept_id = 20;
View created.
SQL> SELECT * from simple_view;
```

EMP_ID	EMP_NAME	SALARY	DEPT_ID
102	Neha Gupta	50000	20
103	Ravi Kumar	45000	20

1.2) Complex Relational View- (Multiple Tables Join)

Definition:

A view is **created using multiple tables, joins, aggregates, or derived columns** is called 'Complex relational view'

Purpose:

It combines data from multiple tables into a single virtual table.

Example-

Consider we have tables 'dept' and 'emp' tables as above-

We create a complex relational view to **show employees with department names**.

```
SQL> CREATE VIEW emp_dept_view AS
2 SELECT e.emp_id, e.emp_name, e.salary, d.dept_name, d.location
3 FROM EMP e
4 JOIN DEPT d
5 ON e.dept_id = d.dept_id;
View created.
SQL> SELECT * from emp_dept_view;
```

EMP_ID	EMP_NAME	SALARY	DEPT_NAME	LOCATION
101	Amit Sharma	40000	HR	Delhi
102	Neha Gupta	50000	IT	Mumbai
103	Ravi Kumar	45000	IT	Mumbai
104	Sunita Mehta	60000	Finance	Pune

2. Object View:

Definition:

A view is said to be object view **if it is created on user-defined object type** is called 'Object view'.

OR

A view that returns **rows (records) as objects of a user-defined object type** is called 'Object view'

Purpose:

It enables object-oriented operations in Oracle.

Example-

To create object view, first **we have to create Object type**. We create object as follow-

```
SQL>CREATE TYPE emp_dept_obj AS OBJECT (
emp_id NUMBER,
emp_name VARCHAR2(20),
salary NUMBER,
dept_name VARCHAR2(20)
);
```

After creating object, we create object view as follow-

```
SQL> CREATE TYPE emp_dept_obj AS OBJECT (
2     emp_id NUMBER,
3     emp_name VARCHAR2(20),
4     salary NUMBER,
5     dept_name VARCHAR2(20)
6 );
7 /
```

Type created.

```
SQL> CREATE VIEW emp_dept_obj_view OF emp_dept_obj
2 WITH OBJECT IDENTIFIER (emp_id) AS
3 SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
4 FROM EMP e
5 JOIN DEPT d ON e.dept_id = d.dept_id;
```

View created.

```
SQL> SELECT * FROM emp_dept_obj_view;
```

EMP_ID	EMP_NAME	SALARY	DEPT_NAME
101	Amit Sharma	40000	HR
102	Neha Gupta	50000	IT
103	Ravi Kumar	45000	IT
104	Sunita Mehta	60000	Finance

Here, object view returns each row as an **object**.

3. Read-only View

Definition:

A view that **cannot be updated** (not allow insert/update/delete) is called 'Read only view'

Purpose:

Read only view **provides data security or simplifies complex query without modifying data.**

Example-

```
SQL> CREATE VIEW high_salary_view AS
2 SELECT emp_name, salary
3 FROM EMP
4 WHERE salary > 50000
5 WITH READ ONLY;
```

View created.

```
SQL> SELECT * FROM high_salary_view;
```

EMP_NAME	SALARY
Sunita Mehta	60000

```
SQL> UPDATE high_salary_view SET salary = 65000 WHERE emp_name='Sunita Mehta';
UPDATE high_salary_view SET salary = 65000 WHERE emp_name='Sunita Mehta'
*
```

```
ERROR at line 1:
ORA-01733: virtual column not allowed here
```

Here, Attempt to update is failed.

4. Materialized View

Definition:

A view that stores **physical copy of data of one or more tables in the database** is called as “Materialized view”.

Purpose:

Materialized view improves performance for frequent queries. Can be refreshed periodically.

Example-

```
SQL> CREATE MATERIALIZED VIEW emp_material_view AS SELECT e.emp_id,e.emp_name,e.salary,d.dept_name FROM emp e JOIN dept d ON e.dept_id=d.dept_id;

Materialized view created.

SQL> SELECT * FROM emp_material_view;
```

EMP_ID	EMP_NAME	SALARY	DEPT_NAME
101	Amit Sharma	40000	HR
102	Neha Gupta	50000	IT
103	Ravi Kumar	45000	IT
104	Sunita Mehta	60000	Finance

Here, materialized view stores aggregated data physically for better performance.

5. View with WITH CHECK OPTION

Definition:

A view contains WITH CHECK OPTION that ensures any insert/update through the view satisfies the view's WHERE condition.

Purpose:

This view enforces data integrity through the view.

Example-

```
SQL> CREATE VIEW it_emp_view AS
 2  SELECT * FROM EMP
 3  WHERE dept_id = 20
 4  WITH CHECK OPTION;

View created.

SQL> INSERT INTO it_emp_view VALUES (105, 'Rohan Singh', 55000, 20);

1 row created.

SQL> INSERT INTO it_emp_view VALUES (106, 'Anita Jain', 50000, 10);
INSERT INTO it_emp_view VALUES (106, 'Anita Jain', 50000, 10)
      *
ERROR at line 1:
ORA-01402: view WITH CHECK OPTION where-clause violation
```

Here, it ensures data modifications respect to the view's condition.